INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY
HYDERABAD

IIIT-H

INLP Project- LyricGpt

Interim Report

Arnav Agnihotri

June 12, 2026

Contents

Chapter 1

Introduction

Executive Summary

This report presents the progress made on developing a computational model for generating Hindi song lyrics. The project employs a novel approach by fine-tuning the GPT-2 language model with a custom dataset of Hindi lyrics (written in Latin script) enhanced with audio features and sentiment analysis. The current prototype demonstrates promising results in maintaining rhythmic structure but faces challenges with semantic coherence. This mid-evaluation details our methodology, achievements, challenges, and outlines future directions to improve the system's performance.

1.1 Project Overview

The project aims to create an AI-powered system capable of generating contextually relevant and structurally sound Hindi lyrics based on musical and emotional parameters. By incorporating audio features and sentiment analysis into the language model training process, we hope to create a tool that can generate lyrics appropriate for specific musical contexts.

1.2 Objectives

- Create a comprehensive dataset of Hindi lyrics with associated audio features and sentiment analysis
- Fine-tune a language model to generate Hindi lyrics (in Latin script) that match specified audio features
- Evaluate the quality of generated lyrics based on structural and semantic coherence
- Compare performance across different prompt engineering techniques and model configurations

Chapter 2

Methodology

This chapter outlines the methodology adopted in this project, including the creation of a multi-modal dataset, the selection and fine-tuning of the language model, and the development of prompt engineering strategies to generate meaningful lyrical content.

2.1 Dataset Creation

We have successfully built an extensive dataset consisting of the following components:

- **Hindi song lyrics in Latin script:** We searched the Internet to find a dataset that had 4300+ Hindi songs with their lyrics in Latin script and the English translation of the lyrics along with them. The major feature we wanted in the lyrics was that the lyrics should be well organized in different lines that are well separated by punctuation so that we could add a rhyme scheme to the lyrics without facing any problems. The audio files were extracted using the yt-dlp lib on Python. We then using the Librosa module on Python extracted the important features for the audio that we needed to train the model on like tempo, energy, valence, etc.
- **English song lyrics:** Again, we had problems in looking for a dataset that had English songs with well-separated lyrics but did find one and used the same steps as the Hindi dataset to get the final dataset.
- **Audio features:** Extracted using the Librosa library, which includes:
  - Valence (emotional positivity)
  - Tempo (beats per minute)
  - Danceability
  - Energy
  - Acousticness
  - Instrumentalness

  - Loudness
  - Speechiness
  - Liveness

- **Rhyme scheme extraction:** We also added rhyme scheme analysis to our dataset. For each song, we looked at the last few characters of each line to guess how the rhymes work. If two lines ended similarly, we gave them the same rhyme label (like A, B, C, etc.). This helped us get a pattern like ABAB or AABB for each stanza. We added this info as new columns to the dataset, including a nice formatted version to show which line got which rhyme.
- **Sentiment analysis:** We used the SamLowe/roberta-base-go_emotions model from HuggingFace to analyze the emotional content in English translations of the lyrics. For each song, we took a small chunk of the lyrics and predicted the top 4 emotions (excluding neutral). These emotions were added to the dataset as a new column, letting us see how emotions align with rhyme patterns and song structure.

This multi-modal dataset enables us to explore the relationship between musical features, emotional tone, and lyrical composition.

2.2 Model Selection and Fine-tuning

For the generation task, we selected GPT-2 as our base model due to its strong performance in natural language generation tasks. Although GPT-2 is primarily trained on English corpora, it can effectively model Hindi written in Latin script, making it suitable for our task.

The fine-tuning process involved the following steps:

1. **Preprocessing:** We cleaned the dataset by standardizing character encodings, removing noisy or incomplete lyric lines, and aligning audio features with their corresponding lyrics. Numerical features were normalized to a consistent scale, such as min-max scaling, to improve prompt interpretability.
2. **Prompt Construction:** Structured prompt templates were created to embed audio features directly into the input text. The final prompt format was:

```
[FEATURES] tempo: 0.56, valence: 0.33, energy: 0.21, ...
   Generate
hindi lyrics [LYRICS]
```

3. **Tokenizer and Model Setup:** We modified the GPT-2 tokenizer to include additional special tokens like `[FEATURES]` and `[LYRICS]`. The model's embedding layer was resized accordingly to accommodate the new vocabulary.
4. **Fine-tuning:** The model was trained using the HuggingFace Trainer API with the AdamW optimizer (learning rate = 5×10^{-5}). Gradient accumulation was used to

manage memory efficiency. We also applied early stopping based on validation loss to avoid overfitting.

5. **Evaluation:** Since lyric generation is a subjective task, evaluation was done qualitatively. Generated outputs were checked for coherence, fluency, and emotional alignment with the input audio features. We also monitored loss and perplexity to ensure training stability.

2.3 Prompt Engineering Approach

Prompt engineering was crucial in providing the model with enough context to produce musically and emotionally aligned lyrics. We experimented with several formats and finalized the following structure:

- **Audio Features:** A structured list of comma-separated features prefixed by `[FEATURES]`, for example:

```
[FEATURES] tempo: 112.5, valence: 0.22, danceability: 0.15, ...
```

- **Generation Cue:** A fixed phrase `Generate hindi lyrics` added after the features to explicitly instruct the model.
- **Generation Token:** The token `[LYRICS]` indicates where the model should start generating lyrical content.

During inference, the prompt was tokenized and passed into the model. Sampling strategies such as nucleus sampling (top-p = 0.9), top-k = 40, and temperature = 0.5 were used to balance creativity and relevance. We also enforced `no_repeat_ngram_size = 2` to avoid repetition and generated multiple sequences using `num_return_sequences = 3`.

A regex-based filtering method was applied to extract only the content after the `[LYRICS]` token. Among the outputs, the one with sufficient word count and coherence was selected as the final result. This structured approach enabled the model to produce lyrics that were both musically grounded and thematically relevant.

2.4 Summary

In this chapter, we presented the methodology used in our project. We described the construction of a multi-modal dataset that combines lyrical data with corresponding audio features and emotional cues. We explained the selection and customization of the GPT-2 model for lyric generation, including modifications to the tokenizer and the introduction of special prompt tokens. We also discussed our approach to fine-tuning the model using structured input formats and sampling-based generation techniques. These methodological foundations enable the generation of emotionally and musically grounded Hindi lyrics that align with the input audio characteristics.

Chapter 3

Current Results

This section contains the milestones that we have achieved till now in our attempt to make a lyric generation model.

3.1 Achievements

Right now, the model's doing a decent job. Here's what it's pulling off:

- **Rhythmic sense is on point:** The lyrics it's generating flow well and match the tempo and beat patterns from the audio features.
- **Structure makes sense:** Most outputs follow typical Hindi song formats — like proper verses and sometimes even chorus-style sections.
- **Listens to the vibe:** If you change features like tempo or valence, you can actually see the mood and style of the lyrics shifting to match. But the proportion of change is yet to be perfected in the correct direction.

3.2 Limitations

While the current version of our model shows promising results, a few limitations are still there:

- **Semantic coherence is still weak:** Even though the structure and rhythm are mostly fine, the meaning doesn't always flow smoothly. Lines sometimes feel disconnected, and the overall theme can get lost.
- **Language model bias:** Since GPT-2 is trained mostly on English data, generating high-quality Hindi lyrics (even in Latin script) is still a bit challenging. Some outputs feel unnatural or grammatically odd.
- **Feature influence is inconsistent:** Audio and sentiment features are being used in the prompt, but their effect on the generated lyrics isn't always reliable. Some features

like tempo or valence show clear influence, others don't.

3.3 Example of a Lyrics Generated

Jeevan jeev mein joh jeena hai Kya haal kya kuch bhi hawa haan Jaun jaane koi kahin kyun jaye hain Jaa haath kabhi karne kare kheen karde haaye Jave haathon mehndi haai jaan Jahan jahan mehar haa jayega Jahaan jahaans mehera karo jaa Jeez jeez me jisne jaisi jawaani Jekhne toh bharne ki dhadkan jahiyan Jism mehta na jaana kahan Kuch kho gaya toot jaaye hum Kise kha gayi toote jaise hum Jiske

3.3.1 Link to Music Generated from Lyrics Generated

Link to Sample Song from generated Lyrics

Chapter 4

Future Work

4.1 Model Exploration

To address the current limitations, we plan to:

- Evaluate alternative language models better suited for multilingual or specifically Hindi text generation.
- Consider models with larger parameter counts to capture more nuanced language patterns.
- Explore models that have been pre-trained on multilingual datasets.

4.2 Feature Engineering

We plan to create a set of feature weights according to the significance of features in lyrics:

- Try different sets of weights for different features and run trials.
- Find the best set of feature weights that result in coherent, rhythmic, and semantically accurate lyrics.

4.3 Comparative Analysis

We will conduct a systematic comparative study of:

- Different base models and their performance on Hindi lyrics generation.
- Various prompt engineering techniques.
- The impact of different normalization approaches for audio features.
- The effects of varying training dataset sizes.

- Comparative studies on the impact of different components of the semantic coherence score.

4.4 English Lyrics Generation

As a complementary direction, we will develop a parallel system for English lyrics generation using the same methodology. This will provide:

- A baseline for comparison with the Hindi lyrics generation.
- Insights into language-specific challenges.
- A framework for quantitative evaluation of cross-lingual performance.

4.5 Hyperparameter Tuning

In order to achieve the best results we will also perform an extensive hyperparameter tuning on tunable features to find the best combination for lyric generation.

4.6 Final End Product

As part of our future roadmap, we aim to develop a comprehensive pipeline that takes an audio file as input and produces its corresponding lyrics as output, leveraging extracted audio features.

The planned pipeline will involve the following key steps:

1. **Audio Chunking:** The input audio will be split into manageable chunks of 15–20 seconds duration. This enables finer control over temporal features and aligns well with how lyrical segments typically progress in music.
2. **Feature Extraction:** For each audio chunk, we will extract relevant audio features such as MFCCs, chroma features, tempo, and rhythm-based descriptors. These features will serve as the contextual input for the lyric generation model.
3. **Sequential Lyric Generation:** Lyrics will be generated in a sequential manner, where the model receives:
   - The current chunk's audio features, and
   - The last few words or lines of the previously generated lyrics.

   This recursive design ensures continuity and coherence in the generated lyrics, mimicking how human-written lyrics often evolve with the flow of the music.

Chapter 5

Technical Implementation Details

We will discuss our current approach to lyrics generation with rhythmic and signal awareness. This section includes technical details on how exactly we have currently tried to fine tune GPT-2 to generate Hindi lyrics.

5.1 Implementation Architecture

Our implementation utilizes a custom training pipeline built with PyTorch and the Hugging Face Transformers library. The architecture consists of:

- **LyricsDataset Class:** A custom dataset implementation that formats input data by combining audio features and lyrics with specific formatting tokens.
- **LyricsTrainer Class:** A custom trainer that implements semantic coherence improvements during the training process.
- **Generation Function:** A specialized text generation function designed to create lyrics based on provided audio features.

5.1.1 Data Formatting

The system processes input data using the following structure:

Listing 5.1: Code snippet of prompt engineering

```
prompt = f"[FEATURES] duration_ms: {row['duration_ms']}, tempo: {row['tempo']}, "
prompt += f"energy: {row['energy']}, loudness: {row['loudness']}, "
prompt += f"danceability: {row['danceability']}, speechiness: {row['speechiness']}, "
prompt += f"acousticness: {row['acousticness']}, instrumentalness: {row['instrumentalness']}, "
prompt += f"liveness: {row['liveness']}, valence: {row['valence']}, "
```

```
prompt += f"Sentiments: {row['Sentiment']}, "
prompt += "[LANG:ROMANIZED_HINDI] [LYRICS] "
```

This ensures the model can distinguish between feature information and the target lyrics.

5.2 Semantic Coherence Enhancement

We have tried to implement an approach to enhance semantic coherence during training:

Listing 5.2: Code snippet of semantic coherence

```
def _calculate_semantic_coherence(self, generated_text):
    # Extract only the lyrics part (after [LYRICS])
    lyrics_match = re.search(r'\[LYRICS\](.*)', generated_text, re.
       DOTALL)
    if not lyrics_match:
        return 0.0

    lyrics = lyrics_match.group(1).strip()

    # Split lyrics into lines
    lines = [line.strip() for line in lyrics.split('\n') if line.
       strip()]
    if len(lines) <= 1:
        return 0.0

    # Basic coherence check:
    # 1. Minimum word count per line
    # 2. Avoid excessive repetition

    # Check word count
    word_count_score = sum(len(line.split()) >= 4 for line in lines)
        / len(lines)

    # Check for repetition
    unique_lines = len(set(lines)) / len(lines)

    # Normalize score
    return min(max((word_count_score + unique_lines) / 2, 0), 1)
```

This method calculates a coherence score based on two primary metrics:

- Maintaining adequate word count per line
- Avoiding excessive repetition of identical lines

The resulting score is used as a regularization term during training, with a weight factor (currently set to 0.3) to balance between standard language modeling and semantic coherence objectives.

5.3 Model Configuration

5.3.1 Special Tokens

We extended the GPT-2 tokenizer with special tokens to better handle our specific formatting:

Listing 5.3: Code snippet of special tokens

```
special_tokens = {
    'pad_token': '[PAD]',
    'additional_special_tokens': ['[FEATURES]', '[LYRICS]', '[LANG:
       ROMANIZED_HINDI]'],
}
tokenizer.add_special_tokens(special_tokens)
model.resize_token_embeddings(len(tokenizer))
```

These tokens help the model distinguish between feature descriptions and lyrical content during both training and generation.

5.3.2 Training Parameters

The fine-tuning process used the following parameters:

- Base model: GPT-2 (standard)
- Learning rate: 5e-5
- Epochs: 5
- Batch size: 4
- Maximum sequence length: 512 tokens
- Optimizer: AdamW with warmup schedule
- Semantic coherence weight: 0.3

5.4 Generation Strategy

For generating new lyrics, we implemented a specialized generation function with the following parameters:

Listing 5.4: Code snippet of generation technique

```
def generate_lyrics(model, tokenizer, audio_features, max_length
   =200,
                    temperature=0.7, top_k=40, top_p=0.9):
```

Notable generation parameters include:

- Temperature: 0.7 (controls randomness)
- Top-k: 40 (limits consideration to top 40 token probabilities)
- Top-p: 0.9 (nucleus sampling threshold)
- No-repeat-ngram-size: 2 (prevents immediate repetition of bigrams)

Chapter 6

Conclusion

The project has made significant progress in creating a specialized dataset and developing a functional prototype for Hindi lyrics generation. The current model shows promising results in capturing structural and rhythmic elements of lyrics while responding to specified audio features. Our innovative approach to enhancing semantic coherence during training demonstrates an understanding of the challenges in cross-lingual text generation.

To advance the project toward its final goals, we will work on:

- **Model Enhancement:** Explore multilingual models or models specifically pre-trained on Hindi text to improve semantic coherence. Consider models like MuRIL (Multilingual Representations for Indian Languages) or IndicBERT that may have stronger representations of Hindi linguistic patterns.
- **Semantic Coherence Refinement:** Enhance the current semantic coherence calculation with more sophisticated linguistic features specific to Hindi, possibly incorporating dependency parsing or other structural analysis tools.
- **Evaluation Framework:** We will try to develop a more structured evaluation methodology to assess both technical metrics and subjective quality of generated lyrics. This should include human evaluations from Hindi speakers to judge cultural and linguistic appropriateness.

The promising initial results and our innovative approach to semantic coherence suggest that with these refinements, the system could become a valuable tool for music creators looking for AI assistance in the lyric-writing process. The techniques developed here may also contribute to the broader field of cross-lingual creative text generation.

Chapter 7

References

1. Stanford CS224n Final Project - Lyrics Generation.
2. SamLowe. roberta-base-go_emotions. HuggingFace.
   `https://huggingface.co/SamLowe/roberta-base-go_emotions`
3. Librosa: Python Library for Audio and Music Analysis.
   `https://librosa.org/`
4. yt-dlp: A youtube-dl fork with additional features and fixes.
   `https://github.com/yt-dlp/yt-dlp`
5. GPT-2, RoBERTa, and IndicBERT model documentation. HuggingFace.
   `https://huggingface.co/models`